

# 技术报告

## 大力水手

|          |                                                                     |
|----------|---------------------------------------------------------------------|
| 比赛       | FlagOS Open Computing Global Challenge / LongContext-ICL-Annotation |
| 报告版本     | 2026-04-11 版本                                                       |
| 当前正式最好成绩 | <b>v48 = 74.35</b> (截至 2026-04-11 已返回的官方评测结果)                       |
| 评测口径     | 竞赛结果由线上预测表现与技术方案质量共同决定，本文聚焦方案设计、复现路径与关键实验结论。                        |
| 当前工作重点   | 以长上下文 ICL 为主线，围绕 Task7 进行可复现、可审计、超窄修补式持续提分，并以当前正式最好成绩为稳定基线持续优化。     |

**报告摘要。** 本队基于官方提供的 Qwen3-4B 长上下文基线，构建了一条“可部署、可复现、可小步快跑”的竞赛工作流。系统层面保留统一的八任务推理框架，方法层面遵循“先做单任务真实增益，再做 changed-row 审计收缩风险，最后用极窄 factual patch 吃剩余分数”的原则。实践结果表明，Task7 这条线在 2026-04-09 至 2026-04-11 期间形成了稳定的线上转化链路：从 `v34 = 73.18` 持续提升到 `v48 = 74.35`，成为全队当前最可靠的增分主轴。

报告范围。 本文重点覆盖任务理解、系统设计、复现流程、关键技术路线与版本演进证据，目标是完整展示本队在长上下文 ICL 自动标注场景中的工程实现与方法选择依据。文中既关注最终结果，也关注这些结果为何能够以稳定、可复查的方式被反复得到。

说明：本文中成绩与版本关系均按仓库内已留档的正式结果、构包记录和阶段工件进行整理。

## 1. 比赛任务理解与方案约束

LongContext-ICL-Annotation 的核心目标，是评估大语言模型在超长上下文条件下，使用 in-context learning 完成自动数据标注的能力。官方数据说明指出，大多数任务要求最小 ICL 上下文长度为 30K tokens，Task8 为较短上下文的算子生成特例；所有任务均采用固定测试划分，以保证提交之间的公平性与可复现性。因此，我们从一开始就把“单次高分”拆成两个同等重要的工程目标：一是跑得出来，二是以后还能稳定复现。

| Task | 任务类型                       | 最小上下文 | 测试规模 | 策略关注点              |
|------|----------------------------|-------|------|--------------------|
| 1    | closest integers           | 30K   | 500  | 优先做本地规则直解，减少无谓生成。  |
| 2    | count nouns / verbs        | 30K   | 500  | 结构化计数和依存规则比纯生成更稳定。 |
| 3    | collatz conjecture         | 30K   | 500  | 尽量转为可验证的局部求解逻辑。    |
| 4    | code string concat         | 30K   | 500  | 关注格式约束和答案抽取。       |
| 5    | tweet sadness detection    | 30K   | 500  | 偏向检索式长上下文拼接和标签稳态。  |
| 6    | MNLI same genre            | 30K   | 500  | 规则约束与生成混合，防止风格漂移。  |
| 7    | Jeopardy answer generation | 30K   | 500  | 开放答案最敏感，是后期主要提分来源。 |
| 8    | kernel generation          | 16K   | 166  | 更依赖检索与输出包装质量。      |

我们对比赛约束的理解主要有三条。第一，所有预处理、上下文构建和示例选择逻辑，都必须能在技术报告

中清楚交代；这意味着任何“靠手工记忆的调参”都不算完成。第二，长上下文不是装饰，而是方法的一部分，所以我们明确区分 baseline、long\_context、task-specific retrieval 等 profile，并把它们绑定到脚本和环境变量。第三，真正能稳定提分的，不是同时改很多东西，而是围绕当前正式最好包做单任务替换与小步验证。

**本队的执行原则。** 除了 Task7 在后期成为主要增分点外，其余任务默认以“保底、低波动、少动”为先。只要某条新方法无法形成明确的离线证据和线上转化，我们宁可回到正式基线继续做小修补，也不扩大改动面。

**方法边界。** 本队所有规则增强、答案约束与后处理步骤，均以官方提供的任务输入和公开可核验资料为基础，并以脚本化方式固化在仓库中；对于 Task7 的少量事实修补，也保留了对应的核题依据、构包脚本和 changed-row 工件，以确保能够复查和复现。

**运行框架。** 本队所有正式运行流程均通过 `bootstrap\_h20.sh`、`src/llm\_config.yaml` 与 FlagScale 服务启动路径组织，长上下文推理、环境变量管理和任务执行入口都围绕这一主线展开。

## 2. 系统架构、环境与复现流程

### 2.1 代码结构

仓库采用“统一数据入口 + 单任务策略函数 + 独立构包脚本 + 产物留档”的结构。核心入口为 `src/main.py`

和 `src/run\_all\_tasks.py`：前者负责单任务读取、示例选择、提示构建、调用模型和输出 JSONL；后者负责一次性运行全部八个任务并打包。方法逻辑集中在 `src/method.py`，其中保存了上下文预算、profile、Task7 rerank、author projection、稳定性门、Task2/Task6 规则增强等主要策略。

## 2.2 部署与推理环境

本队沿用官方 README 推荐的 Qwen3-4B 作为主模型，并按照 YaRN 配置扩展长上下文。推理时使用 vLLM 兼容的 `POST /v1/completions` 接口，实际部署配置写入 `src/llm\_config.yaml`，再通过 FlagScale 启动服务。为了保证换机可复现，我们把关键路径都写成环境变量而不是硬编码，主要包括模型路径、tokenizer 路径、服务 URL、端口和 completion seed。

| 环节     | 本队做法                                                                                          |
|--------|-----------------------------------------------------------------------------------------------|
| 模型权重   | 下载 `Qwen/Qwen3-4B` 到本地目录，并同步设置模型与 tokenizer 路径。                                               |
| 长上下文配置 | 在 `config.json` 中启用 YaRN rope scaling，将原始位置编码扩展到竞赛所需长上下文。                                     |
| 推理服务   | 通过 FlagScale 读取 `src/llm_config.yaml` 启动服务，对外提供标准 completions API。                            |
| 单任务运行  | 使用 `python main.py --task_id N` 跑 smoke 或局部复现。                                                |
| 整包运行   | 使用 `python run_all_tasks.py --output_dir ...` 生成 `openseek-1` 至 `openseek-8` 的 JSONL 与 zip 包。 |

## 2.3 可复现实验命令

我们的本地复现流程尽量只保留必要命令。例如，设置环境变量后用 FlagScale 启动服务；然后在本仓库 `src/` 目录运行任务脚本；最后用构包脚本替换目标任务生成新的任务版本。对于 Task7 的后期 workflow，则额外加入 changed-row summary、candidate smoke、verified patch notes 和 online readout 等工件。这样做的好处是，即使隔天再看，也能从文件名和时间戳迅速还原当时的决策链。

从工程视角看，这部分最重要的价值在于“可复现性”。我们把环境说明、模型加载、推理脚本、补丁构建、烟测和版本证据都显式保存在仓库中，而不是散落在临时命令历史里。这样做既方便阶段复盘，也能让整条方法线在不同时间点被重复验证。

示例命令：``python run_all_tasks.py --output_dir ../outputs/baseline_submission``

示例命令：``python src/build_task7_v49_fact2_candidate.py``

示例命令：``python src/check_task7_v49_candidate_smoke.py``

与运行直接相关的环境配置、部署脚本、推理入口和补丁构建脚本都与本文描述保持一致，便于从报告跳转到代码实现。

## 3. 方法设计：从统一长上下文基线到任务定制增强

---

### 3.1 统一长上下文 ICL 主线

在 ``method.py`` 中，我们把所有任务共享的基础能力统一到 `profile` 配置里，包括上下文预算、chat example budget、example pool limit、检索开关以及少量 task-specific 例外。这样做的原因是，比赛的核心不是单个 prompt，而是“在长上下文限制下如何稳定组织示例”。对于没有特别强结构先验的任务，系统默认先从官方 examples 池中截取一定规模的候选，再根据任务特性进行重排或筛选，最后拼接进统一的 prompt 模板。

### 3.2 能本地求解的任务先本地求解

本队没有把所有任务都一股脑扔给生成模型。像 Task1、Task2、Task3 这类存在明确规则或局部算法结构的任务，我们优先在 ``solve_task_locally`` 或依赖策略模块中处理；只有当无法稳定本地决策时，才回到模型生成。这个设计在两个层面上有效：一是减少 token 消耗和长上下文浪费，二是降低开放生成造成的波动。

### 3.3 检索、重排与二级候选

对于更依赖语义匹配的任务，我们使用轻量检索或重排来优化 ICL 示例顺序。例如 Task5/Task6 使用聊天检索或 genre-aware 重排，Task7/Task8 使用 lexical retrieval。需要强调的是，我们并没有追求特别复杂的向量检索系统，而是优先使用仓库内即可复现的规则化重排方式，因为比赛时间短、线上验证成本高，可维护性比复杂度更重要。

### 3.4 Task7 的特殊性

Task7 是开放式问答生成，答案形式不稳定、别名多、长上下文提示对最终输出的影响非常明显，因此它也是最容易既带来增益又引入大面积副作用的任务。我们的后期增分全部围绕 Task7 展开，但执行上遵守两条硬约束：第一，不在多任务同时大改；第二，每轮只围绕少量 Task7 变更行推进，并保留完整 changed-row 证据。这个约束帮助我们z把方法收益与副作用分开观察，而不是让多条改动轴线相互掩盖。

**核心工程判断。** 比赛后半段真正有效的不是“继续找一个更强的宏大方法”，而是围绕当前 best

official baseline 做超窄、可验证、对副作用有约束的局部修补。也正因为如此，我们后期每个版本

通常只改 Task7 的极少数行，但线上仍能持续小幅提升。

## 4. Task7 提分主线：从方法增益到超窄修补

Task7 的提分过程并非单一脚本直接完成，而是经历了“主增益线成型 -> changed-row 审计 -> 强负例回退 -> 超窄 factual patch”四个阶段。其中最关键的一步，是把 author projection 从研究态转成稳定的主线版本，即 `anchor1\_stability\_gate`。这个版本的设计重点不在于多找候选，而在于先冻结同一轮候选，再用重复稳定性和双向 pairwise 去约束晋升，尽量避免把一次 listwise 运气当成真实收益。

### 4.1 稳定性门的作用

稳定性门只在 author 相关 bucket 中触发，复用同一轮生成得到的 primary、secondary 和 author projected candidate list，不重新采样 example pool。它先分别计算 baseline judge winner 和 anchor1 judge winner；若两者不同，则要求 anchor1 的结果在重复 judge 中三次完全一致，并在 pairwise `[baseline, anchor1]` 与 `[anchor1, baseline]` 两个顺序下都获胜，才允许覆盖 baseline。否则统一回退到 baseline winner。这个机制直接解决了早期 replay parity 对不齐、author catalog 为空时容易出现的单次误判问题。

## 4.2 关键离线验证

围绕稳定性门，我们保留了明确的 target-side 证据链。三条关键样本 `3deb...`、`88c5...`、`02e748...` 在三种 seeds 下都满足预期，其中 `88c5...` 在 seeds `2027` 与 `2028` 上明确体现为 gate rescue：离线旧预测仍偏向 `john updikey`，而稳定性门后的输出能稳定落到 `John Dos Passos`。同时，在 36-row 校准集上，stability-gate 的 replay changed count 为 4，优于 baseline 的 6 与 anchor1 的 7，说明它不仅保留 target gain，也压缩了非目标漂移。

| 阶段     | 代表版本    | 线上成绩           | 主要动作                                                            |
|--------|---------|----------------|-----------------------------------------------------------------|
| 主增益成型  | v34     | 73.18          | 实现 author projection 稳定性门，把 research state 转成稳定可复用的 Task7 主线版本。 |
| 强负例回退  | v35     | 73.45          | 对 changed rows 做 triage，仅回退被 AB/BA 双向 pairwise 明显击败的新答案。        |
| 极窄事实修补 | v36     | 73.60          | 从 manual review 中挑出高置信 clue-level factual rows，形成 6 行补丁包。       |
| 连续超窄修补 | v37-v48 | 73.70<br>74.35 | -> 坚持每轮只改极少数高置信行，持续保留已验证 target rescue，并把正式基线不断前推。              |

## 4.3 为什么这种路线能持续提分

我们总结出几个关键经验。其一，方法级大改只有在能先形成 packageable 增益时才值得做，`v34` 正是这样一个例子。其二，一旦新方法线上转正，就不应该马上再开另一条大线，而应该先把 changed rows 作为主要审计对象。其三，对于开放答案任务，很多失分并不是“模型完全不会”，而是答案形式、别名或 clue 对齐不够硬，因此极少量 factual patch 往往比继续改 judge layout 更便宜、更安全，也更容易线上转化。



## 5. 线上成绩演进、工件体系与可复现性

### 5.1 官方成绩演进

截至 2026-04-11，本队已把正式线上最好成绩推进到 **74.35**。根据 `official\_score\_ledger\_2026-04-09.json`，后期连续有效的版本链为：`v36 = 73.6`、`v37 = 73.7`、`v38 = 73.78`、`v39 = 73.78`、`v40 = 73.85`、`v42 = 73.98`、`v43 = 74.05`、`v44 = 74.10`、`v45 = 74.15`、`v46 = 74.23`、`v47 = 74.30`、`v48 = 74.35`。这条轨迹说明，后期策略并不是偶然命中，而是形成了可以重复使用的增分机制。

| 版本                                                                 | 基线  | 官方分数  | 说明                                      |
|--------------------------------------------------------------------|-----|-------|-----------------------------------------|
| v34                                                                | v30 | 73.18 | 稳定性门首次转正，author projection 从研究状态进入正式包线。 |
| v35                                                                | v34 | 73.45 | changed-row 审计后回退 69 个强负例，明显提升线上分数。     |
| v36                                                                | v35 | 73.60 | 首轮极窄 factual patch 转正，验证了“少改高置信行”的路线。   |
| v37-v48 滚动前推 73.70 -> 74.35 持续构建 Task7-only 小补丁版本，并严格以当前正式最好包为新基线。 |     |       |                                         |

### 5.2 工件留档体系

为了确保每次提分不是“只剩一个压缩包”，我们为每轮关键动作都生成工件：`verified\_patch\_notes` 记录人工核题与外部证据；`candidate\_build` 记录变更行与理由；`candidate\_smoke` 记录结构完整性、关键保底行和新补丁行检查；`online\_readout` 记录官方成绩与基线关系；`official\_score\_ledger` 记录整个版本谱系。这套工件体系是后期连续提分的重要基础，因为它降低了遗忘成本，也避免了在多个相近版本之间混淆。

### 5.3 当前稳定状态

截至本文写作时，本队已经形成一条清晰的正式版本推进链：每次只在当前正式最好基线上做 Task7 的极小范围高置信修补，完成结构烟测、changed-row 审计和工件留档后再进入评测。这个过程保证了版本关系、证据口径和线上结果之间的一致性，也让后续优化能够继续建立在稳定正式基线之上，而不是回到早期版本重复试错。

复现结论。 从工程上看，本队当前最成熟的不是某个单独脚本，而是一整套“产生版本、验证版本



构包、记录、再从正式最好基线继续”的循环。只要保留这套循环，后续即使换人继续，也能沿着同一

证据口径推进。

## 6. 局限性与后续方向

### 6.1 工程组织与版本留档

本文涉及的工程实现主要分布在 `src/`、`outputs/` 和 `docs/phase-summaries/` 三部分：`src/` 提供方法实现、构包脚本和验证脚本；`outputs/` 保存正式版本包、任务输出和 work logs；`docs/phase-summaries/` 记录阶段性总结与线上成绩 readout。这样的组织方式使方法逻辑、实验工件与阶段结论能够相互对应，便于沿着时间线回溯整条优化路径。

### 6.2 当前局限

首先，Task7 后期的很多修补依赖人工核题与外部资料交叉验证，这虽然在线上很有效，但对自动化程度仍有提升空间。其次，部分早期任务改动虽然离线看起来有收益，但没有稳定转化到线上，因此我们后期选择更保守的单任务小补丁路线。最后，当前流程已经能稳定支撑版本推进，但在自动化打通、依赖统一和流水线封装方面仍有继续工程化的空间。

### 6.3 后续工作计划

短期内，我们仍会保持“当前正式最好基线 + 超窄 Task7 factual patch”的策略，不重开大范围方法实验。中期来看，如果 Task7 的超窄修补收益开始衰减，我们会重新审视其他任务是否存在同样安全的小范围收益点，但仍坚持单任务替换与完整工件留档。长期来看，我们计划把目前散落在若干脚本里的构包、验证和提分经验进一步产品化为统一流水线，以降低后续维护成本。

### 6.4 结论

本队在本次比赛中的关键收获，不只是把分数推到 74.35，更重要的是沉淀出了一条可复现的长上下文 ICL

竞赛工程路径：以统一基线托住整体稳定性，用任务定制策略做局部增益，再用极窄、可审计、可解释的补丁推动线上结果持续上升。我们相信，这种“把研究想法做成工程闭环”的方式，比单次幸运命中更有价值，也更符合比赛对技术报告与复现性的要求。

主 要 依 据 的 本 地 材 料 包 括 ： `README.md` 、 `src/README\_local.md` 、 `data/README\_zh.md` 、 `outputs/work\_logs/official\_score\_ledger\_2026-04-09.json` 、 `docs/phase-summaries/2026-04-09-Task7 连续提分经验总结.md` 以及 2026-04-10 至 2026-04-11 的若干 Task7 online readout 与 verified patch notes。